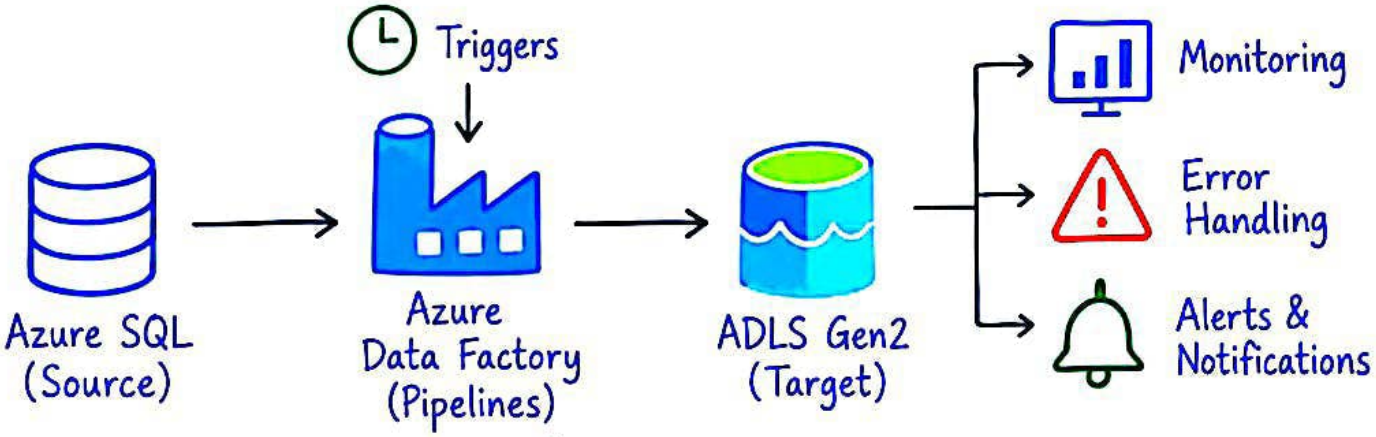
NEERAJ SINGH
DEVOPS

30 Days Learning Challenge

Azure Data Engineering

ADF Triggers, Monitoring & Error Handling

Build reliable and automated data pipelines for real-world scenarios



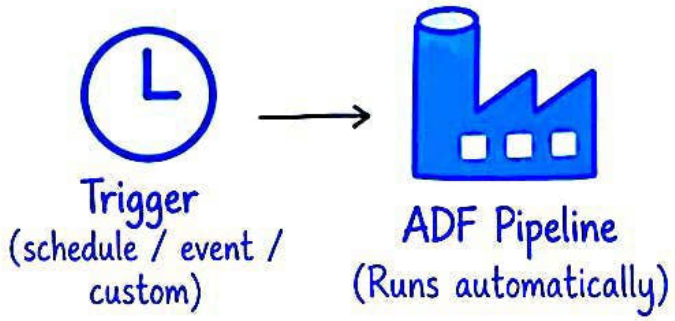
From Zero to Azure Data Engineer

ADF Triggers - Overview

Triggers automatically start your ADF pipelines based on a schedule, event or custom condition.

1 What is an ADF Trigger?

- A trigger defines when a pipeline should run.
- It automates pipeline execution without manual intervention.
- Helps to build reliable and scheduled data workflows.



2 Types of ADF Triggers

1. Schedule Trigger



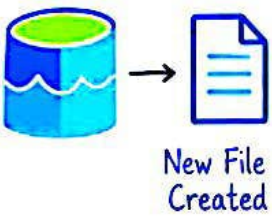
- Runs pipeline at specific time or intervals.
- Supports recurring schedules (minutes, hours, days).
- Most commonly used trigger.

2. Tumbling Window Trigger



- Runs pipeline for fixed, non-overlapping time windows.
- Useful for historical data processing and backfills.
- Supports window dependency and retry.

3. Storage Event Trigger



- Triggers pipeline when a file is created or updated in storage.
- Supports Azure Blob Storage and ADLS Gen2 events.
- Useful for real-time and event-driven data pipelines.

4. Custom Event Trigger



- Triggered by custom events using Azure Event Grid.
- Useful for external systems, applications or custom workflows.
- Helps to integrate ADF with other Azure services.

Schedule vs Tumbling Window Trigger

Both triggers are time-based, but they work differently and are used for different scenarios in Azure Data Factory.

1 Key Differences

Feature	Schedule Trigger	Tumbling Window Trigger
Execution Model	Runs pipeline at specific time or regular intervals	Runs pipeline for fixed, non-overlapping time windows
Use Case	Daily/Hourly/Weekly pipeline runs	Processing historical data, backfills, time-partitioned data
Window	No time window	Fixed start and end time for each window
Backfill (Rerun)	Manual rerun only	Supports backfill for past windows easily
Dependency	No window dependency	Supports dependency between windows
Retry	Supports retry	Supports retry for failed windows
Typical Usage	Regular data loads, reports	Incremental loads, historical data, time-based processing

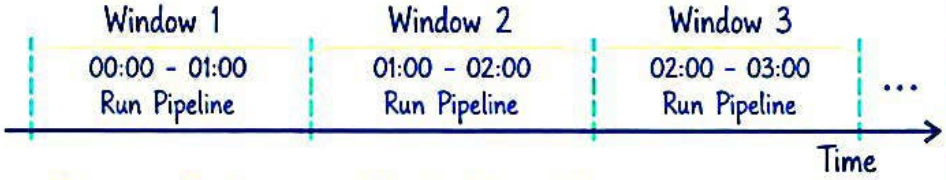
2 Schedule Trigger Example



Common Schedule Options

- Every hour (0 * * * *)
- Every day at 2 AM (0 2 * * *)
- Every week (0 2 * * 0)
- Custom intervals (every 15 mins)

3 Tumbling Window Trigger Example



Window Configuration

- Window size: 1 hour / 1 day etc.
- Start time: 00:00
- Frequency: 1 hour
- Max concurrency: 1
- Dependency: Optional

- Processes fixed, non-overlapping time windows.
- Can rerun failed or past windows (backfill).
- Useful for incremental loads and historical data.

ADF Monitoring - Overview

Monitor pipeline, activity and trigger runs to check status, performance and failures in Azure Data Factory.

1 What Can We Monitor?

- Pipeline runs (overall status)
- Activity runs (each activity status)
- Trigger runs (scheduled/event triggers)
- Duration, start time, end time
- Input/Output details
- Errors and logs

ADF Monitor

Pipeline runs

Activity runs

Trigger runs

Name	Status	Start time	Duration
Load_Customers	Succeeded	10:00 AM	5 mins
Load_Orders	Failed	10:15 AM	2 mins
Load_Products	Succeeded	11:00 AM	4 mins

2 Monitoring Components

Pipeline Runs

- Overall pipeline status
- Start time, end time
- Parameters used
- Duration and run ID

Activity Runs

- Each activity status
- Input / Output details
- Duration
- Error message
- Retries

Trigger Runs

- Trigger execution history
- Triggered time
- Status (Succeeded/Failed)
- Pipeline run details
- Manual or scheduled

Integration with Azure Monitor

- Send logs to Log Analytics
- Create alerts for failures
- Track metrics and trends
- Integrate with Action Groups

3 ADF Monitoring Flow

Trigger (Schedule/Event) → ADF Pipeline (Activities) → Pipeline & Activity Logs (Status, Errors, Metrics) → Azure Monitor (Log Analytics) → Alerts & Notifications (Email / Action Group / Logic Apps)

4 Common Status Types

Succeeded	Failed	In Progress	Cancelled
Activity/pipeline completed successfully	Activity/pipeline failed	Currently running	Manually cancelled

5 Key Benefits

- Identify failures quickly
- Analyze performance and duration
- Debug pipeline errors
- Set up alerts and notifications
- Ensure reliable and production-ready pipelines

ADF Error Handling & Retry

Handle failures, configure retries, and build fault-tolerant pipelines for reliable data processing.

1 Common Failures in ADF

- Source or destination connection failed
- File not found in storage
- Authentication / permission issues
- Schema mismatch
- Timeout due to large data volume
- Activity failure (Lookup, Copy, ForEach etc.)
- Network or transient failures

2 Retry Policy in ADF

- Retry count: Number of times to retry a failed activity.
- Retry interval: Time gap between retries (in seconds).
- Applies to activities like Copy, Lookup, Web, etc.
- Useful for transient failures (network, service unavailability).

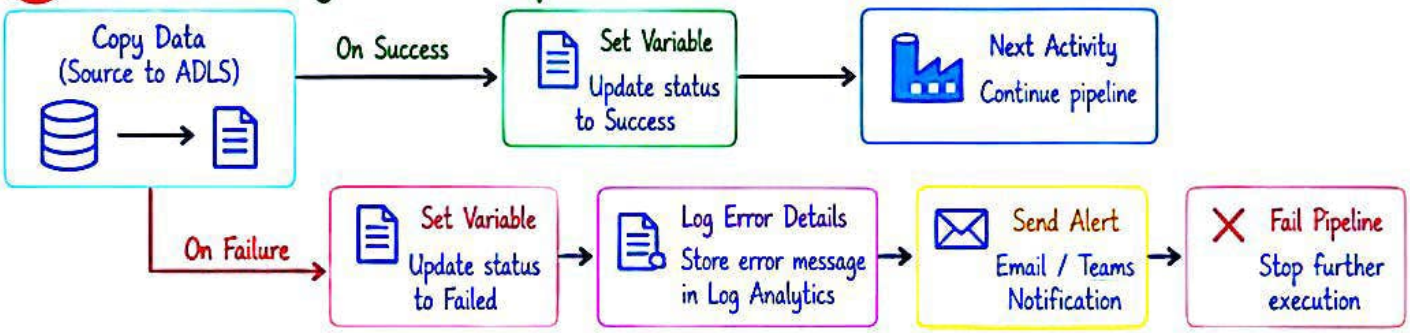
Example:
Retry count = 3
Retry interval = 30s

3 Activity Dependency Conditions

- Define what should happen if a previous activity succeeds, fails, skips or completes.
- Used to build error handling logic and fallback paths.

Succeeded	Failed	Skipped	Completed
Run next activity	Run error handling activity	Run alternative path	Run next activity

4 Error Handling Flow Example



5 Best Practices

- Use retry for transient errors only.
- Set appropriate timeout for large data loads.
- Use dependency conditions for error handling.
- Log errors to Log Analytics or storage for troubleshooting.
- Send alerts using Azure Monitor / Logic Apps.
- Design idempotent pipelines to handle retries safely.
- Test failure scenarios before moving to production.

6 Common Interview Questions

1. What is the retry policy in ADF?
2. Which activities support retry and timeout?
3. How do you handle failures in an ADF pipeline?
4. What are activity dependency conditions?
5. How do you send email alerts on pipeline failure?
6. How do you ensure idempotent data loads when retrying?

Alerts & Notifications in ADF

Use Azure Monitor and Action Groups to get notified when ADF pipelines fail or face issues.

1 What are Alerts?

- Alerts notify you when a pipeline, activity or trigger fails.
- Helps to proactively identify and resolve issues.
- Can send notifications via email, SMS, Teams or webhook.

2 ADF Alert Workflow



3 Steps to Configure Alerts

1. Enable Diagnostic Settings

- Send ADF logs to Log Analytics Workspace.

2. Create Alert Rule

- Select failure conditions.
- Use Log Analytics queries or built-in metrics.

3. Create Action Group

- Add notification channels (Email, Teams, SMS etc.).
- Map the action group to the alert.

4. Link to ADF

- Apply the alert rule to ADF resources.
- Test by triggering a pipeline failure.

4 Common Alert Conditions

- Pipeline run failed
- Activity run failed (e.g., Copy, Lookup, ForEach)
- Trigger run failed
- Pipeline run duration exceeded threshold
- High failure rate in a specific time window
- Integration runtime issues
- Data flow failures
- Custom log-based alerts using Log Analytics queries

5 Example Alert Rule

Condition: Pipeline run status = Failed
Scope: Azure Data Factory
Action: Send Email to team

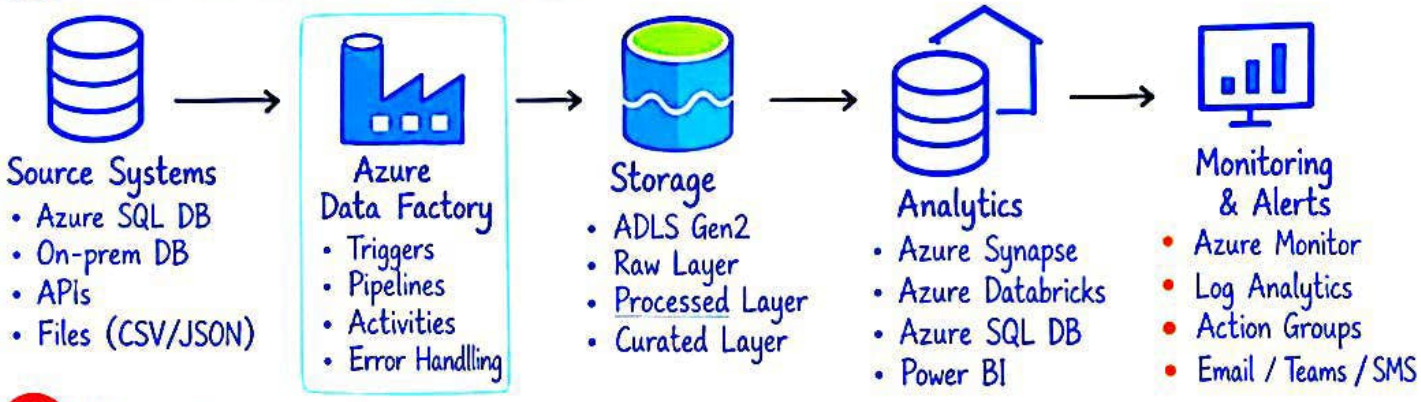
6 Best Practices

- Enable diagnostic settings for ADF.
- Use appropriate alert thresholds.
- Notify the right team with clear messages.
- Monitor both pipeline and activity runs.
- Test alerts regularly.
- Use Teams or Slack integration for quick response.

Production Pipeline Design in ADF

End-to-end design for a reliable, scalable and production-ready data pipeline.

1 End-to-End Architecture



2 Key Components

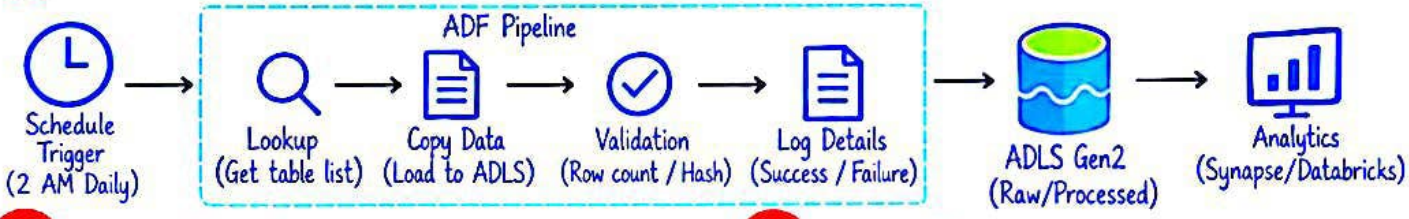
Triggers
- Schedule Trigger
- Tumbling Window
- Event Trigger
- Custom Trigger

Pipelines
- Control flow activities
- Copy data activity
- Parameters & Variables
- Dynamic expressions

Data Storage
- ADLS Gen2 (Raw/Processed)
- Partitioning strategy
- Folder structure by date
- Data validation (count/hash)

Monitoring
- Pipeline & activity runs
- Logs and metrics
- Alert rules and notifications
- Track failures and retries

3 Production Pipeline Flow



4 Best Practices

- Use parameterized and metadata-driven pipelines.
- Implement proper folder structure and partitioning.
- Enable retry policies and failure notifications.
- Validate data (row count, schema) before and after load.
- Use idempotent design to avoid duplicate data.
- Monitor pipeline performance and optimize.
- Secure connections using Managed Identity / Key Vault.
- Maintain logs and audit for compliance.

5 Benefits

- Automated and reliable data ingestion.
- Handles failures with retries and alerts.
- Scalable for multiple tables and sources.
- Monitoring and logging for better visibility.
- Supports incremental and historical loads.
- Production-ready and maintainable solution.

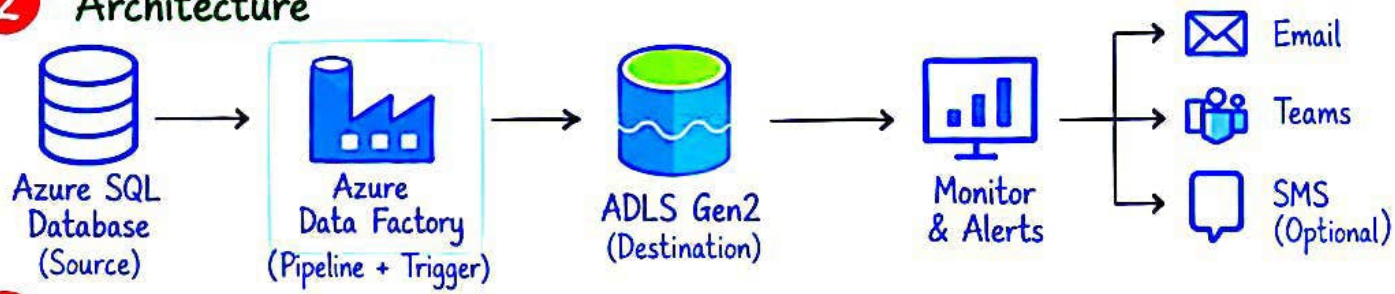
Hands-on Challenge & Interview Questions

Build a real Azure Data Factory pipeline, monitor it and configure alerts.

1 Hands-on Challenge

Scenario: Copy sales data from Azure SQL Database to ADLS Gen2 every day at 2 AM, with failure notifications.

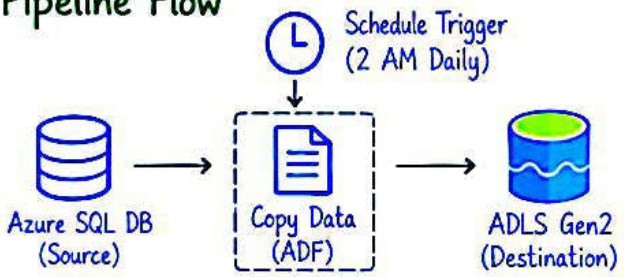
2 Architecture



3 Step-by-Step Tasks

1. Create a new ADF pipeline
2. Create linked services (Azure SQL DB → ADLS Gen2)
3. Create datasets for source and destination
4. Add Copy Data activity and configure mappings
5. Create a Schedule Trigger (daily at 2 AM, IST)
6. Configure retry (e.g., 3) and timeout (e.g., 30 mins)
7. Publish and start the trigger
8. Test a failure scenario (e.g., wrong connection)
9. Check pipeline runs in Monitor
10. Configure Azure Monitor alert and Action Group (email)

Pipeline Flow



Expected Output

- Data copied from Azure SQL DB to ADLS Gen2
- Pipeline runs successfully on schedule
- Logs visible in ADF Monitor
- Email notification received on failure

4 Common Interview Questions

1. What are the different triggers in ADF?
2. What is the difference between Schedule and Tumbling Window Trigger?
3. How do you monitor pipeline and activity runs in ADF?
4. How do you handle failures and configure retry in ADF?
5. How do you send email notifications on pipeline failure?
6. How do you design an idempotent ADF pipeline?
7. What are activity dependency conditions?
8. How do you backfill historical data using Tumbling Window Trigger?

5 Key Takeaways

- Use the right trigger based on the business need.
- Monitor pipeline, activity and trigger runs.
- Configure retry, timeout and dependency conditions.
- Integrate with Azure Monitor for alerts.
- Follow idempotent design and proper logging.
- Build reliable and production-ready pipelines.
- Always test failure scenarios.